

Processzusok, szálak/fonalak, processzus létrehozása/befejezése, processzusok állapotai, processzus leírása. Ütemezési stratégiák és algoritmusok köteget, interaktív és valós idejű rendszereknél, ütemezési algoritmusok céljai. Kontextus-csere

OPERÁCIÓS RENDSZER - BEVEZETŐ

- Az operációs rendszer egy szoftver, ami kezeli a számítógép hardverét és szoftverét, hogy könnyen használhassa a felhasználó
- feladatai:
 - **felhasználó kényelmének és védelmének biztosítása**
 - könnyen kezelhető funkciók az emberi felhasználók számára
 - hatékony jogosultságkezelés
 - biztosítja futás közben a használható eljárásokat és függvényeket, miközben védi a futó programok memóriaterületét
 - **rendszer hatékonyságának, teljesítményének maximalizálása**
 - mikor egyszerre több program fut a gépen, az OS gondoskodik róla, hogy mindegyik elegendő erőforrást kapjon és ne zavarják egymást
 - erőforrások hatékony kezelése, belerétve a CPU-t, a memóriát, az I/O (Input/Output) eszközöket, valamint szoftvereket (eljárások, bufferek, üzenetek és szolgáltatások)

PROCESSZUSOK, SZÁLAK/FONALAK

- a processzusokat (folyamatokat) és szálakat az OS kernelje (szoftver) irányítja
 - a CPU-hoz (hardver) tartozó szoftver a kernel

Processzusok

- **a processzus (folyamat) egy végrehajtás alatt lévő program**
 - a program maga egy statikus egység, mely a lemezen tárolódik
 - a processzus egy aktív entitás, mely a memóriában van és állapottal és erőforrásokkal is rendelkezik
 - tehát nem csak a programkódot jelenti, hanem a kód futtatásához szükséges környezetet (állapotot)
- a CPU egyidőben csak egy folyamatot futtat, de gyorsan váltogat a folyamatok között (**Virtuális CPU koncepció**)
 - minden processzus úgy érzékeli, hogy saját CPU-val rendelkezik
 - multiprogramozás - párhuzamosság illúzióját kelti
- egy folyamat **saját címtartománnyal és állapottal** rendelkezik
- **saját címtartomány** a memória egy minimális és egy maximális című helye között lévő szept ez a saját területe ahol írhat és olvashat
 - **programkód:** tartalmazza a végrehajtandó utasításokat
 - **adatterület:** globális, lokális változók, dinamikusan foglalt memóriaterület
 - **verem (stack) memória:** lokális változókat, eljárások hívási adatait és visszatérési címeket tárolja

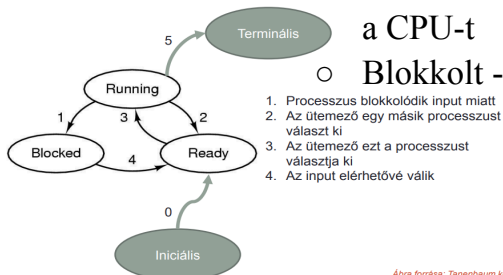
- miket használhat egy folyamat:
 - **regiszterek** (regiszterkészlet)
 - regiszter egy olyan gyors memóriatároló, ami közvetlenül a processzorba van
 - ideiglenesen tárolja az utasításokat, adatokat és a számítási eredményeket, ezzel gyorsítja a processzor számításait
 - **Utasításszámláló** (Program Counter - PC)
 - követi, hogy épp hol tart a program az utasítások végrehajtásában (memóriacímét tárolja a következő utasításnak)
 - Veremmutató
 - mivel a verem dinamikusan változik (nincs egy konkrét memóriacíme) ezért a veremmutató azt mondja meg, hogy hol van a memóriában a verem aktuális teteje
 - egyéb hardverregiszterek
 - pl: I/O, azaz bemeneti/kimeneti műveletek irányítására
 - regiszterek, mely a program futásához szükséges egyéb információk tárolja
 - **erőforrások**
 - megnyitott fájlok
 - gyerekprocesszusok listája
 - szignálkezelők (aszinkron (egy időben futó) események kezelésére szolgáló funkciók)
 - függőben lévő ébresztők (időzített események kezelésére szolgáló mechanizmusok)
- hagyományos OS esetén egyetlen vezérlési szállal rendelkeznek

Processzusok leírása

- **Processzustáblázat**
 - OS egy táblázatban tartja nyilván az összes létező folyamatot, minden bejegyzése egy folyamatot reprezentál
- **PCB** (Process Control Block - Processzus Vezérlő Blokk)
 - a folyamat adatlapja, amely minden fontos információt tárol a kontextuscseréhez és ütemezéshez:
 - **Folyamat azonosítója** (PID) és a szülő azonosítója
 - **Processzus állapota** (futó, futáskész, blokkolt)
 - **Regiszterek mentett értékei** (program számláló, veremmutató, általános regiszterek)
 - **Ütemezési információk** (prioritás, felhasznált CPU idő)
 - **Birtokolt erőforrások**
 - **Memória kezelés** (mutatók a kód-, adat- és veremszegmensére)
 - **Fájlkezelés** (gyökérkönyvtár, akutális könyvtár, stb.)

Processzusok állapotai

- **Új (iniciális):** a processzus létrehozása folyamatba van
- **Futáskész:** a folyamat készen áll a futásra, de ideiglenesen várakozni kell, mert a CPU-t egy másik folyamat már használja
- **Futó:** a folyamat éppen használja a CPU-t
- **Blokkolt (várakozó):** a folyamat valamilyen külső eseményre vár (pl: bilentyűleütés, adatok beolvasása lemezzről, stb), és még vár nem tud továbbhaladni
 - blokkolt állapotról soha nem kerülhet futó állapotra, azaz nem kerülhet rögtön CPU-hoz, mert előbb mindeképp futáskész állapotba kell kerülnie, hogy az ütemező választhassa
- **Befejezett (terminális):** a futás véget ér, befejeződött a folyamat
- **Állapot átmenetek:**
 - Futó -> Blokkolt: a folyamat I/O műveletet kér, vagy erőforrásra vár
 - Futó -> Futáskész: az **ütemező** úgy dönt, hogy lejárt a folyamat időszelete és másnak adja a CPU-t
 - Futáskész -> Futó: az **ütemező** kiválasztja futásra az adott feladatot, neki adja a CPU-t
 - Blokkolt -> Futáskész: bekövetkezik a várt esemény (kész I/O művelet)



Ábra forrása: Tanenbaum könyv

Processzusok létrehozása

- A folyamatok létrehozásának 4 fő oka / eseménye van:
 1. **Rendszerindítás**
 - az operációs rendszer indulásakor számos folyamat jön létre (pl: háttérfolyamatok hálózatkezeléshez, naplózás, stb)
 2. **Folyamatlétrehozó rendszerhívás**
 - egy már futó folyamat kezdeményezheti új folyamat létrehozását
 - szülő - gyermek kapcsolat
 3. **Felhasználói kérés**
 - pl a felhasználó elindít egy programot (pl: parancssorból)
 4. **Köteget feladat indítás (láncbafűzés)**
 - batch rendszereknél
 - amikor az egyik befejeződik és jön a következő
- UNIX rendszerekben a folyamatok létrehozása:
 - két lépés:
 - *fork()* rendszerhívás lemásolja, klónozza a szülő folyamatot (létrejön a gyerek) vagy új programot tölt be
 - majd az *exec()* lecseréli a memóriatartalmat az új programra
 - a létrehozás után a szülő vagy párhuzamosan fut tovább a gyerekekkel vagy megvárja a *wait()* hívással, még a gyerek folyamat befejeződik

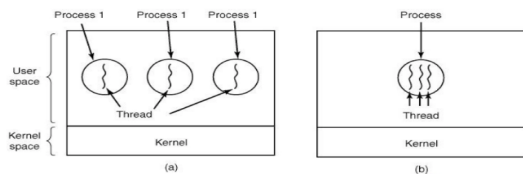
- Létrehozás technikai lépései (UNIX példán keresztül):
 1. **Erőforrás foglalás**
 - OS helyet foglal a memóriában a folyamat leírójának (PCB)
 2. **PCB inicializálás**
 - beállításra kerül a processzus azonosítója (PID), a szülő azonosítója és a kezdeti regiszter értékek
 3. **Címtartomány kialakítása (memóriafooglalás)**
 - lefoglalják a kód és az adat és a verem memória területét
 4. **Futáskész állapotba helyezés**
 - a folyamat bekerül a futáskész listába, várva az ütemezőre, hogy megkaphassa a CPU-t

Processzusok befejezése

- a folyamatok leállítása történhet **önkéntesen** vagy **kényszerítve**
 - **Normális befejezés** (önkéntes)
 - a folyamat elvégezte a feladatát és kilép (pl: *exit* hívással)
 - **Hiba miatti kilépés** (önkéntes)
 - a folyamat hibát észlel (pl: nem talál egy fájlt) és emiatt befejezi a futását
 - **Végzetes hiba** (kényszerített)
 - az operációs rendszer állítja le a folyamatot szabálytalan művelet miatt (pl: nullával való osztás, illegális memóriahozzáférés, stb)
 - **Másik folyamat által leállítás** (kényszerített)
 - egy másik folyamat megöli a folyamatot (*kill* parancs)
 - általában szülő vagy rendszergazda
- a folyamatok befejezésének lépései:
 1. **Gyerekfolyamatok kezelése**
 - bizonyos rendszerekben a szülő összes gyerekfolyamata is automatikusan leáll vagy átkerül más folyamatokhoz
 2. **Erőforrások felszabadítása**
 - a nyitott fájlok lezárása, hálózati kapcsolatok bontása
 3. **Memória felszabadítása**
 - a kód, adat és verem területek visszaadása rendszernek
 4. **PCB törlése**
 - a folyamat kikerül a rendszer táblázataiból (állapota terminális lesz, majd törlődnek)
- **Zombi folyamat**
 - ez egy olyan befejeződött folyamat, amelynek az erőforrásait már felszabadította a rendszer, de a PCB bejegyzése még megvan, mert a szülője még nem olvasta ki a kilépés kódját a *wait()* segítségével
 - zombi állapotra azért van szükség, hogy a szülő is megkaphassa az információt a gyerek kilépésének okáról, még akkor is ha épp mással van elfoglalta mikor a gyerek épp kilép

Vezérlési szálak (szálak, fonalak)

- a szál a végrehajtás legkisebb egysége, amit az OS ütemezni tud
 - egy szál egy processzuson belül kerül végrehajtásra, de külön fogalom a 2
- a szálakat azért használjuk, mert könnyebb és gyorsabb őket létrehozni vagy váltani közöttük, mint a processzusok között, mivel nem kell új címtartományt építeni
- emellett lehetőséget ad a programnak arra, hogy **több tevékenységet végezzen el párhuzamosan egy processzuson belül**, azaz **egy processzuson belül több szál is létezhet**, ezek osztoznak a processzus erőforrásain
 - közös erőforrások: címtartomány (kód és adatok), globális változók, megnyitott fájlok, szignálok
- rendelkezik:
 - **saját utasításszámláló** (nyilvántartja melyik utasítás következik)
 - **saját regiszterkészlet** (aktuális munka változóit tárolja)
 - **saját verem** (a végrehajtási előzmények rögzítéséhez)
 - **saját állapot** (futó, blokkolt)



- a (b) rész több szálát tartalmaz, míg az (a) rész csak egyetlen szálát mutat be
- megvalósítási módok
 - **Felhasználói szintű szálak**
 - a rendszermag (kernel) nem tud róluk, a processzus maga kezeli őket könyvtárakon keresztül
 - saját privát szál táblázattal rendelkezik mindegyik processzus
 - gyors váltás, de ha egy szál blokkolódik (pl: I/O miatt), az egész folyamat (processzus) leáll
 - azért blokkolódik a teljes processzus, mivel a kernel csak egyetlen végrehajtásnak látja az egész processzust, nem tudja, hogy több szál van belül, így pl egy I/O hívásnál az egészet várakozó állapotba teszi
 - **Rendszermag (kernel) szintű szálak**
 - az OS ismeri és külön ütemezi ezeket a szálakat
 - egy szál blokkolódásakor akár ebből akár egy másik processzus szálából választ az ütemező
 - egy globális szál táblázatban tartja nyilván a szálhoz tartozó információkat
 - lassabb váltás (kontextus csere miatt), de egy szál blokkolódása nem állítja le a többi szálát

PROCESSZUS-ÜTEMEZÉS

- amikor több folyamat vagy szál futáskész állapotba van akkor el kell dönteni, hogy ezek közül melyik folyamat kapja meg a CPU-t, azaz kerüljön futási állapotba
 - ezt a döntést az **Ütemező** hozza meg egy **ütemezési algoritmus** alapján
- az ütemezés hatékonysága kulcsfontosságú, mert a CPU sebessége nagyságrendekkel nagyobb, mint a perifériáké, így rossz ütemezés sok üresjáratot okoz
- **Folyamatok viselkedése**
 - **CPU igényes:** hosszú ideig végez számításokat, viszont ritkán vár I/O-ra
 - **I/O igényes:** gyakran blokkolódik I/O művelet miatt, de kevés CPU-t használ
 - a legtöbb folyamat modern rendszerekben ilyen
- Mikor ütemezünk?
 - új folyamat jön létre (a szülő vagy a gyerek fusson?)
 - egy folyamat befejeződik (új folyamatnak kell adni a CPU-t)
 - egy folyamat blokkolódik (pl: I/O művelet miatt, szemafor)
 - I/O megszakítás érkezik (egy várakozó folyamat futáskészé válik)
 - időzítő megszakítás (órajel) érkezik

Ütemezési típusok (Működési módok)

- 2 fő megközelítés van létezik, attól függően, hogy a CPU, hogyan vehető el:
 - **Nem megszakítható (Non-preemptive / nem kooperatív)**
 - a folyamat addig fut, amíg önként le nem mond a CPU-ról, vagy be nem fejeződik
 - régebbi rendszerekre jellemző
 - **Megszakítható (preemptive / kooperatív)**
 - az OS (pontosabban az ütemező) bármikor dönthet úgy, hogy felfüggeszti a folyamatot és egy másik folyamatnak adja a CPU-t (pl: a folyamat időszelete lejárt és másiknak adja)
 - biztonságosabb, mert egy hibás folyamat nem tudja kisajátítani a rendszert

Ütemezési stratégiák és algoritmusok

- az algoritmus célja rendszertípustól változhatnak, de vannak általános szempontok:
 - **Pártatlanság**
 - minden hasonló folyamat hasonló mennyiségű CPU időt kapjon
 - **Ütemezési elvek (policy) betartása**
 - a rendszer előre meghatározott irányelveinek követése, pl: prioritás
 - **Egységes terhelés**
 - a rendszer minden része legyen leterhelve, folyamatos kihasználás
 - amíg CPU számol, menjen az I/O

- Rendszer specifikus célok
 - **Kötegelt (batch) rendszerek**
 - **Átbocsátóképeség (Throughput)**
 - időegység alatt minnél több feladat fejeződjön be
 - kötegelt rendszerekben ez fontosabb mint, hogy egy részfeladat mikor válaszol
 - **Körülfordulási idő (Turnaround time), másnéven áthaladási idő**
 - a beküldés és a befejezés között eltelt idő minimalizálása
 - **CPU kihasználtság**
 - a CPU ne álljon üresen, maximálisan ki legyen használva
 - **Interaktív rendszerek**
 - **Válaszidő (Response time)**
 - a felhasználói kérés és a reakció közötti idő minimalizálása
 - pl: billentyűleütés megjelenítése
 - **Arányosság**
 - a felhasználó elvárásainak megfelelése, azaz a processzusok helyes sorrendjének megválasztása
 - pl: egy mentés lehet hosszabb idejű, de egy betű ne késsen
 - **Valós idejű rendszerek**
 - **Határidők betartása (deadlines)**
 - az adatok feldolgozása egy fix időn belül meg kell történjen
 - **Előrejelezhetőség**
 - pontosan előre jelezhető és szabályos ütemezés szükséges
 - pl: multimédia rendszereknél a minőségromlás elkerülése

Kötegelt (batch) rendszerek

- nincs a felhasználóval interakció
- cél az **átbocsátóképeség** (minnél több feladattal végezzen, egy egységnyi idő alatt) és a CPU maximális kihasználása
- **Sorrendi - FCFS (First Come First Served)**
 - legegyszerűbb algoritmus, a folyamatok érkezési sorrendbe kerülnek a CPU-ra és addig futnak még be nem fejeződnek vagy blokkolódnak
 - **NEM MEGSZAKÍTHATÓ**
 - pl: mint egy sor a pénztárnál, aki előbb állt be azt szolgálják ki először
 - hátránya a konvoj effektus
 - ha egy hosszú, CPU igényes folyamat kerül sorra, az összes rövid folyamat ami mögötte van várakozni kényszer (rontja az átlagos várakozási időt)
- **Legrövidebb feladat először - SJF (Shortest Job First)**
 - mindig azt a folyamatot választja ki amelynek a becsült futási ideje a legrövidebb
 - **NEM MEGSZAKÍTHATÓ**
 - bizonyíthatóan ez adja a minimális várakozási időt, de csak akkor ha induláskor minden feladat egyszerre rendelkezésre áll

- **Legrövidebb maradék idejű - SRTN (Shortest Remaining Time Next)**
 - SJF megszakítható változata
 - azaz ha egy új folyamat érkezik, aminek teljes futási ideje kevesebb, mint a jelenleg futó processzus hátralévő ideje, akkor az ütemező félbeszakítja a futót és az újnak adja a CPU-t
 - pl: főzöl (20 perc van még), de szól a futár, hogy megjött a csomag (2perc átvenni), ekkor átveszed a csomagot, majd folytatod a főzést
 - nehézsége, hogy ismerni vagy becsülni kell a folyamatok futási idejét, ez gyakorlatban nehéz

Interaktív rendszerek

- itt a válaszidő a legfontosabb, azért, hogy a felhasználó ne érezze, hogy a gép gondolkodik / számol
- **Round-Robin (időszelletes ütemezés)**
 - minden folyamat kap egy FIX időszelket (kvantum, 20-50ms általában)
 - túl kicsi akkor sok a kontextus csere (pazarlás)
 - túl nagy akkor romlik a válaszidő (FCFS szerű lesz)
 - cél, hogy a kontextus-csere ideje ne legyen több mint a CPU idő 1%
 - mikor egy futó folyamat időszelke lejár az ütemező megszakítja a futását és a futáskész sor végére teszi, a sor elején lévő folyamatot pedig futtatja
 - pl: Egy tanár körbemegy az osztályban és minden diák kérdésére maximum 1 percet szán, majd megy a következőhöz
- **Prioritásos ütemezés**
 - minden folyamathoz rendelünk egy számot, azaz prioritást
 - ez lehet statikus vagy dinamikus (I/O műveletek esetén főként)
 - az I/O igényesek általában magasabbat kapnak
 - mindig a legmagasabb prioritású folyamat fut
 - pl: kórházban a sürgősségin a súlyosabb sérült előbb kerül sorra, mint aki csak lázas
 - veszélye a **kiéheztetés** (starvation), azaz, hogy az alacsony prioritású folyamatok sosem jutnak CPU-hoz
 - megoldás rá az **aging** (öregítés), azaz minnél tovább vár egy folyamat annál inkább nő a prioritása
 - vagy másik megoldás, hogy minél tovább használja egy folyamat a CPU-t úgy csökken a prioritása
 - vagy maximális időszelket rendelünk egy folyamathoz
- **Többszörös sorok (Multilevel Queues / Feedback Queues)**
 - különböző prioritású sorok vannak, egy folyamat ezek között vándorolhat
 - ha elhasználja egy időszelket a gyors sorba, lejjebb kerül egy alacsonyabb prioritású, de hosszabb időszeltes sorba
 - sorok, másnéven osztályok belülről Round-Robinként működnek
 - prioritásokat időnként igazítani kell, hogy az alsóbb sorok ne haljanak éhen

- **Sorsjáték**
 - folyamatok sorsjegyeket kapnak (minnél nagyobb prioritású annál több jegyet, ezzel statisztikailag több CPU-időt kapnak) és az ütemező sorsol és akié a győztes jegy ő kapja a CPU-t
- **Garantált ütemezés**
 - n felhasználó esetén ígéretet teszünk, hogy mindenki $1/n$ részt kap a CPU-ból
 - követjük a tényleges használatot és mindig azt futtatjuk akinek a legkisebb az aránya
- **Legrövidebb processzus következzon**
 - mivel nem ismerjük előre a hosszt, a múltbeli viselkedésből becsüljük és mindig a legrövidebb fut

Valós idejű (real-time) rendszerek

- nem az a cél, hogy “gyors” legyen, hanem, hogy kiszámítható (determinisztikus)
 - tudnunk kell, hogy a válasz garantáltan megérkezik egy időkorláton belül
- általában nincs is szükség megszakításokra, mivel kis esemény- vagy idővezérelt folyamatokról beszélünk
- típusai
 - **Szigorú (Hard RT)**
 - időkorlát túllépése esetén katasztrófa történik (légzsák nem nyílik ki, robotpilóta)
 - **Toleráns (Soft RT)**
 - a késés ugyan minőségromlást okoz, de nem kritikus (pl: akadozik egy yt videó)
- **Statikus RMS (Rate Monotonic Scheduling)**
 - a prioritásokat az alapján osztjuk ki, hogy amelyik gyakrabban fut az kap nagyobb prioritást
- **Dinamikus EDF (Earliest Deadline First)**
 - mindig az a folyamat kapja a CPU-t, amelyiknek a legközelebb van a határideje
 - prioritások futás során folyamatosan változnak, emiatt 100%-os kihasználtság is lehetővé válik
- **Eseményvezérelt (aperiodikus)**
 - külső véletlenszerű megszakításokra (pl: érzékelőre) reagál a rendszer azonnal
- **Idővezérelt (periodikus)**
 - egy belső órajel alapján fix időközönként ellenőrzi az állapotokat (pl: 10ms-enként hőmérsékelt ellenőrzés)
 - **ütemezhetőség feltétele**
 - több periodikus esemény esetén a rendszer akkor ütemezhető, ha a CPU kihasználtsága nem lépi túl a 100%-ot

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq 1$$

- C_i a folyamat számítás ideje
- P_i a folyamat periódusideje
- pl: 2 feladat 50-50% CPU igény esetén pont jó, de ha már 51-51% esetén a rendszer fizikailag képtelen ellátni

- **pivot-inverzió**
 - valós idejű rendszereket megbénító hiba
 - lényege, ha van egy M magas, egy K közepes és egy A alacsony prioritású folyamat
 - A lefoglal egy közös erőforrást (pl: egy szemaforral zárolt adatot)
 - megérkezik M aki szintén ezt az erőforrást kéri, de neki várnia kell A -ra
 - majd jön K is akinek nincs szüksége az erőforrásra, de mivel nagyobb a prioritása A -nál, ezért átveszi a CPU-t
 - ennek az eredménye, hogy M a legfontosabb várakozik, mert A nem tudja befejezni a munkát és ezzel elengedni az erőforrást K miatt
 - megoldás rá a **Prioritás-öröklés**
 - lényege, hogy amíg egy alacsony prioritású folyamat (pl: A) olyan erőforrást használ, amire egy magasabb prioritású (pl: M) folyamatnak is szüksége van, akkor addig az alacsony prioritású (A) megkapja a magasabb prioritású prioritását (örökli)
 - ezzel elkerülhető, hogy kiszorítsa őt egy közepes (K) folyamat

KONTEXTUS-CSERE

- az a művelet, amely az operációs rendszer elmenti egy éppen futó folyamat állapotát (kontextusát), majd betölti egy másik korábban felfüggesztett folyamat mentett állapotát, hogy az folytathassa a futását
 - lehetővé teszi, hogy több program fusson egyszerre
 - míg a csere zajlik a CPU nem végez hasznos munkát a felhasználó számára
- bekövetkezhet, ha:
 - megszakítás érkezik (órjel, I/O, stb)
 - rendszerhívás esetén (pl: blokkolódik az épp futó folyamat, mert fájlt akar olvasni)
 - ütemezési döntés nyomán (pl: lejár az időszetele)
- elmentett állapot, kontextus tartalma:
 - **Programszámláló (PC - Program Counter)**
 - éppen következő utasítás memóriacíme
 - **Processzor regiszterek**
 - összes munkaregiszter aktuális tartalma
 - **Veremmutató (stack pointer)**
 - a folyamat saját vermenek aktuális helye
 - **Memóriakezelő egység (MMU) állapota**
 - oldaltáblák és szegmenstáblák
 - **Processzor állapot-szavak (PSW)**
 - állapotjelző bitek (pl: túlsordulás, előjel)
- pl: P1 folyamattól a P2 folyamathoz kerül a CPU
 - először a P1 kontextusa lementésre kerül
 - majd a P2 folyamat korábban mentett kontextusa kerül betöltésre, így onnan folytathassa ahol abbahagyta

- a kontextus-csere egy **költséges és nehéz folyamat** (ezért találták ki a szálakat, mert ott a címtartomány közös és sokkal gyorsabb a szálak közötti váltás, mint a folyamatok közötti)
 - tisztán adminisztrációs idő
 - regiszterek mentése és betöltése alatt nem fut a felhasználói kód
 - cache hatás
 - CPU gyorsítótárba kezdetben még a régi folyamat adatai vannak, az új folyamatnak még be kell laknia a cachet (hideg cache)
 - kezdetben lassabb futást eredményez
 - TLB (Translation Lookaside Buffer) flush
 - a virtuális címfordítást segítő táblát, a TLB-t gyakran kell újítani a váltáskor, ez drága művelet